

CaptionAI: Credit Card Approval Prediction

Project Description:

Credit Card Approval Prediction is a machine learning-based web application that predicts whether a customer's credit card application will be approved or rejected based on the applicant's details. The system uses historical data and machine learning algorithms to analyze various factors such as income, employment status, age, credit history, and other financial information.

The application provides users with a simple and interactive web interface where they can enter applicant information and receive instant predictions. The project is built using Python, Flask, and Machine Learning libraries such as Scikit-Learn, Pandas, and NumPy.

The main objective of this project is to automate the credit approval process, reduce manual work, improve decision-making speed, and assist financial institutions in evaluating customer applications efficiently.

Scenarios

Scenario 1:

A bank employee enters the details of a salaried customer with a good credit history and high income. The system analyzes the information and predicts that the credit card application is likely to be approved.

Scenario 2:

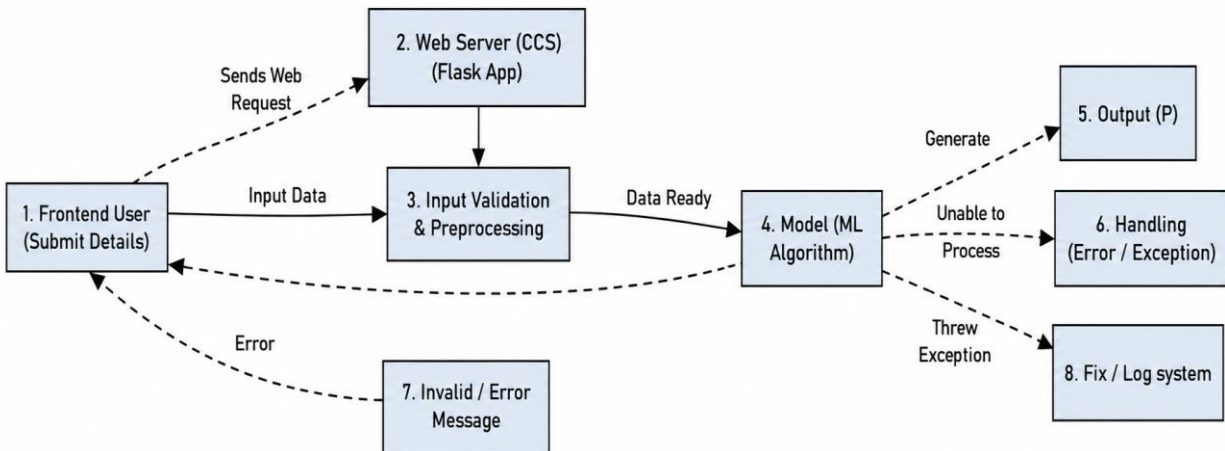
A customer with low income and poor credit history applies for a credit card. The model evaluates the details and predicts that the application may be rejected.

Scenario 3:

A financial institution uses the application to quickly screen multiple applications and make faster decisions regarding credit card approvals.

Technical Architecture:

Technical Architecture:



Description: System Architecture

The Credit Card Approval Prediction system follows a simple machine learning-based architecture consisting of four major components:

1. User Interface (Frontend)

The frontend is developed using HTML, CSS, and JavaScript. Users enter applicant information such as age, income, employment status, and other financial details through a web form.

2. Flask Backend

The Flask application receives the user input, validates the data, processes it, and sends it to the machine learning model for prediction.

3. Machine Learning Model

The trained machine learning model analyzes the applicant's information and predicts whether the credit card application will be approved or rejected.

4. Prediction Result

The prediction result is returned to the Flask application and displayed on the user interface.

Pre-requisites:

- **Flask Framework Knowledge:** Flask Documentation
- **Machine Learning Fundamentals:** Scikit-Learn Documentation
- **Data Analysis and Processing:** Pandas Documentation

- **Numerical Computing:** NumPy Documentation
- **HTML, CSS, and JavaScript Skills:** W3Schools HTML/CSS/JavaScript Tutorials
- **Python Programming Proficiency:** Python Documentation
- **Version Control with Git:** Git Documentation
- **GitHub Repository Management:** GitHub Documentation
- **Development Environment Setup:** Visual Studio Code Documentation
- **Model Serialization and Loading:** Joblib Documentation
- **Web Application Deployment (Optional):** Render Documentation

Project Workflow

Milestone 1: Model Selection and Architecture

- **Activity 1.1:** Collect and analyze the Credit Card Approval dataset and identify important features affecting approval decisions.
- **Activity 1.2:** Research and select an appropriate Machine Learning algorithm for credit card approval prediction.
- **Activity 1.3:** Define the system architecture, detailing interactions between the frontend, Flask backend, and machine learning model.
- **Activity 1.4:** Set up the development environment by installing the required Python libraries and dependencies.

Milestone 2: Core Functionalities Development

- **Activity 2.1:** Develop the applicant data input form to collect user information such as age, income, education, employment status, and credit history.
- **Activity 2.2:** Implement data preprocessing techniques including data cleaning, encoding, and feature scaling.
- **Activity 2.3:** Train and evaluate the machine learning model for predicting credit card approval.
- **Activity 2.4:** Implement the prediction module to classify applications as Approved or Rejected.

Milestone 3: App.py Development

- **Activity 3.1:** Develop the main application logic in app.py and establish routes for handling user input and prediction requests.
- **Activity 3.2:** Load the trained machine learning model and integrate it with the Flask application.
- **Activity 3.3:** Process user input data and generate real-time prediction results.

Milestone 4: Frontend Development

- **Activity 4.1:** Design and develop a responsive user interface using HTML, CSS, and JavaScript.

● **Activity 4.2:** Create dynamic templates using Flask's `render_template` to display prediction results.

● **Activity 4.3:** Implement form validation and user-friendly navigation for better user experience.

Milestone 5: Testing and Deployment

● **Activity 5.1:** Perform functional testing and verify prediction accuracy.

● **Activity 5.2:** Test the application using different applicant data scenarios.

● **Activity 5.3:** Deploy the application locally or on a cloud platform such as Render.

● **Activity 5.4:** Prepare project documentation and demonstration video.

Milestone 5: Deployment

- **Activity 5.1:** Prepare the application for local deployment by configuring the server environment and ensuring all dependencies are correctly installed.
- **Activity 5.2:** Deploy the application publicly using Ngrok to expose the local Flask server over a secure public URL.

Milestone 1: Model Selection and Architecture

In this milestone, we focus on selecting an appropriate Machine Learning model for predicting credit card approval decisions. This involves analyzing the dataset, identifying important features, and choosing a suitable classification algorithm that provides accurate predictions.

Activity 1.1: Collect and Prepare the Dataset

Before training the machine learning model, a suitable dataset containing applicant information is required.

Step 1 — Collect the Dataset

Download the Credit Card Approval dataset from Kaggle or another reliable source.

Step 2 — Analyze the Dataset

Examine the dataset and identify important features such as:

- Gender
- Age
- Education Level
- Annual Income
- Employment Status

- Credit History
- Approval Status

Step 3 — Handle Missing Values

Remove or replace missing values to improve model performance.

Step 4 — Encode Categorical Features

Convert categorical data into numerical values using encoding techniques.

Step 5 — Perform Feature Scaling

Normalize or standardize numerical data for better prediction performance.

Activity 1.2: Select the Machine Learning Model

Research different classification algorithms and select the most suitable model.

Algorithms considered:

- Logistic Regression
- Decision Tree Classifier
- Random Forest Classifier
- Support Vector Machine

The model with the highest accuracy is selected for the final application.

Activity 1.3: Define the Architecture of the Application

• Draft an Architectural Diagram:

Create a visual representation of the application architecture, including components such as the frontend (HTML, CSS, JavaScript), Flask backend, and Machine Learning model integration.

• Detail Frontend Functionality:

Outline how users interact with the application by entering applicant information such as:

- Gender
- Age

- Education Level
- Annual Income
- Employment Status
- Credit History

The user submits the details through a web form and receives an instant prediction result.

• **Outline Backend Responsibilities:**

Specify how the Flask backend:

- Handles incoming POST requests.
- Validates and sanitizes user input.
- Preprocesses the input data.
- Sends the processed data to the trained machine learning model.
- Returns the prediction result to the frontend.

• **Describe Machine Learning Integration:**

Define how the backend:

- Loads the trained model (model.pkl) using Joblib.
- Preprocesses user input data.
- Passes the data to the prediction model.
- Receives the prediction output.
- Displays whether the credit card application is **Approved** or **Rejected**.

Activity 1.4: Set Up the Development Environment

• **Install Python and Pip:**

Ensure Python 3.10+ is installed along with pip.

CreditCardApprovalPrediction/

```
|
|
|— app.py
|
|— model.pkl
|
|— requirements.txt
|
|— README.md
|
|— templates/
|   |— index.html
|
|— static/
|   |— style.css
|
|— dataset/
|   |— credit_card_data.csv
```

Milestone 2: Core Functionalities Development

Activity 2.1: Develop Core Prediction Features

Implement the following core functionalities:

- **Applicant Data Entry**

Collect applicant information through a user-friendly web form.

- **Data Preprocessing**

Perform:

- Data cleaning
- Feature encoding

- Feature scaling
- Credit Card Approval Prediction

Use the trained Machine Learning model to predict whether the application will be approved or rejected.

- Result Generation

Display the prediction result instantly on the web page.

Activity 2.2: Implement the Flask Backend

Define Routes in Flask:

Set up routes in `app.py` for:

- Home page
- Prediction endpoint

Process User Input:

- Create a form in `index.html`.
- Collect applicant details.
- Validate all fields before processing.

Integrate Machine Learning Model:

- Load `model.pkl`.
- Preprocess user data.
- Generate predictions using the model.
- Display the prediction result.

Milestone 3: App.py Development

Milestone 3 focuses on creating and refining the core logic of the `app.py` file, which serves as the backbone of the Credit Card Approval Prediction application. In this milestone, Flask routes are

created, the machine learning model is integrated, and prediction functionality is implemented and tested.

Activity 3.1: Writing the Main Application Logic in app.py

Define the Core Routes in app.py

Home Page Route

The home page route renders the main user interface.

```
@app.route("/")
```

```
def home():
```

```
    return render_template("index.html")
```

Prediction Route

```
@app.route("/predict", methods=["POST"])
```

```
def predict():
```

```
    gender = request.form["gender"]
```

```
    age = request.form["age"]
```

```
    education = request.form["education"]
```

```
    income = request.form["income"]
```

```
    employment = request.form["employment"]
```

```
    credit_history = request.form["credit_history"]
```

```
    prediction = model.predict(input_data)
```

```
    return render_template(
```

```
        "index.html",
```

```
        prediction_text=prediction
```

```
)
```

Set Up Route Handlers for Each Feature

For the /predict route:

- Read user input from the web form.
- Validate all input fields.
- Preprocess the input data.
- Send the processed data to the machine learning model.
- Return the prediction result to the user.

Define Data Preprocessing Helpers

Input Validation

Ensure that:

- No field is left empty.
- Numerical values are within acceptable ranges.
- Invalid values are rejected.

Feature Encoding

Convert categorical values into numerical values.

Example:

Gender:

Male = 1

Female = 0

Credit History:

Good = 1

Bad = 0

Data Scaling

Normalize numerical values before passing them to the machine learning model.

Integrate the Machine Learning Model

Load the trained model:

```
import joblib
```

```
model = joblib.load("model.pkl")
```

Generate Prediction

```
prediction = model.predict(input_data)
```

Application Routes Summary

Home Route

/ → Renders index.html

Responsibilities of app.py

- Load the trained machine learning model.
- Handle user requests.
- Validate and preprocess input data.
- Generate predictions.
- Display prediction results.
- Handle exceptions and errors

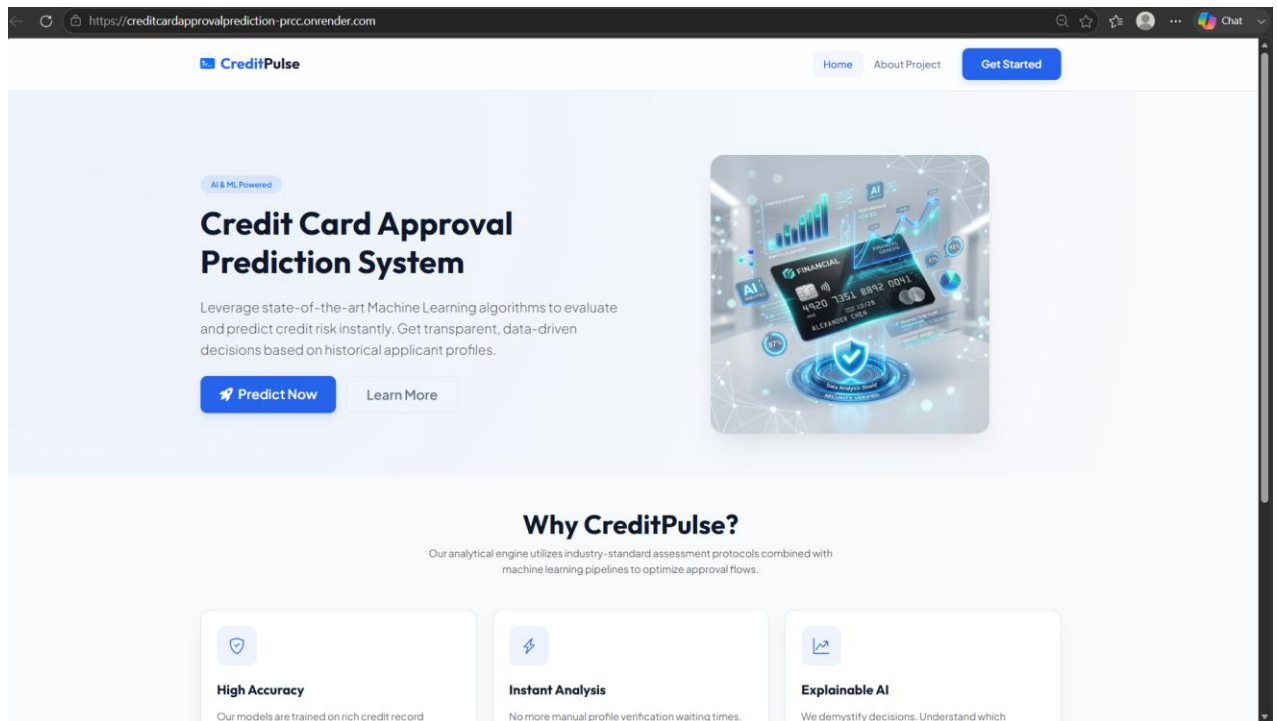
Milestone 4: Frontend Development

Milestone 4 focuses on developing a simple, user-friendly, and responsive interface for the Credit Card Approval Prediction application. The frontend allows users to enter applicant information and view prediction results instantly.

Activity 4.1: Designing and Developing the User Interface

Set Up the Base HTML Structure

- Develop the main HTML file (index.html) as a single-page application.
- Include:
 - Project title and logo
 - Applicant details form
 - Prediction result section
- Use semantic HTML elements to make the application organized and accessible.



Design a Responsive Layout Using CSS

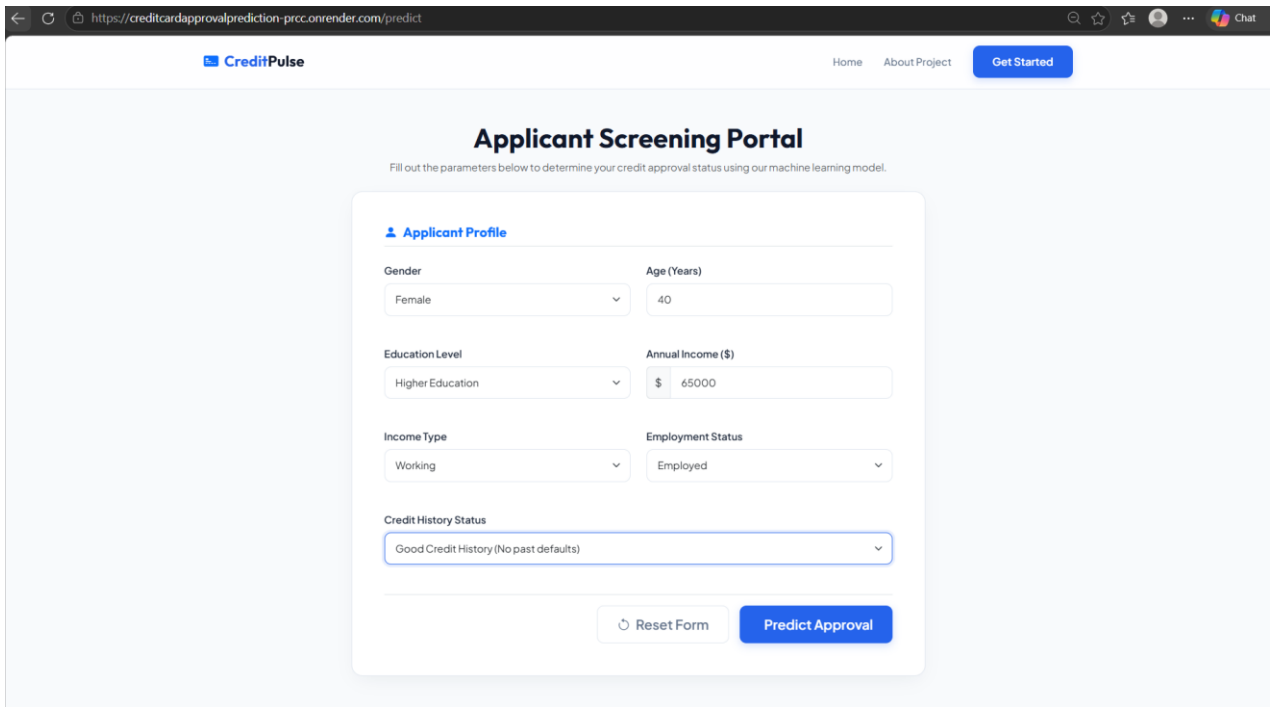
- Create style.css for styling.
- Design a clean and responsive interface.
- Use Flexbox and Grid layouts.
- Add media queries to support:
 - Desktop
 - Tablet
 - Mobile devices.

Create UI Components

Applicant Input Form

The form includes:

- Gender
- Age
- Education Level
- Annual Income
- Employment Status
- Credit History



The screenshot shows a web browser window with the URL <https://creditcardapprovalprediction-prcc.onrender.com/predict>. The page is titled "CreditPulse" and has a navigation bar with "Home", "About Project", and a "Get Started" button. The main heading is "Applicant Screening Portal" with a subtext: "Fill out the parameters below to determine your credit approval status using our machine learning model." Below this is a form titled "Applicant Profile" with the following fields:

- Gender:** A dropdown menu with "Female" selected.
- Age (Years):** A text input field containing "40".
- Education Level:** A dropdown menu with "Higher Education" selected.
- Annual Income (\$):** A text input field with a dollar sign icon and "65000".
- Income Type:** A dropdown menu with "Working" selected.
- Employment Status:** A dropdown menu with "Employed" selected.
- Credit History Status:** A dropdown menu with "Good Credit History (No past defaults)" selected.

At the bottom of the form are two buttons: "Reset Form" and "Predict Approval".

Predict Button

A button to submit applicant information.

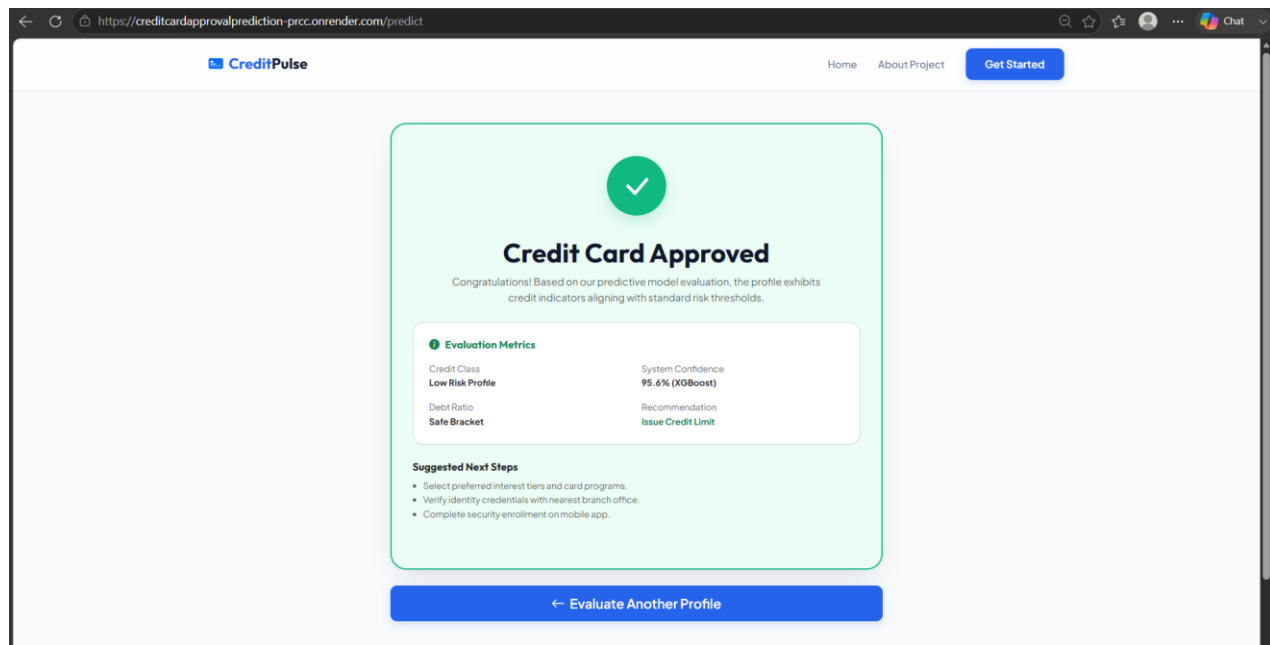
Result Card

Displays:

Credit Card Approved

or

Credit Card Rejected



Activity 4.2: Creating Dynamic Content Rendering

Handle Form Submission

- Submit applicant information to the Flask backend.
- Process user input without reloading the page (optional).

Render Results Dynamically

Display:

- Prediction result
- Status message

Restore Form State

Provide:

- Reset button
- Clear all fields functionality.

Milestone 5: Deployment

Milestone 5 focuses on deploying the Credit Card Approval Prediction application and testing its functionality.

Activity 5.1: Preparing the Application for Local Deployment

Create Virtual Environment

```
python -m venv myenv
```

```
myenv\Scripts\activate
```

```
pip install -r requirements.txt
```

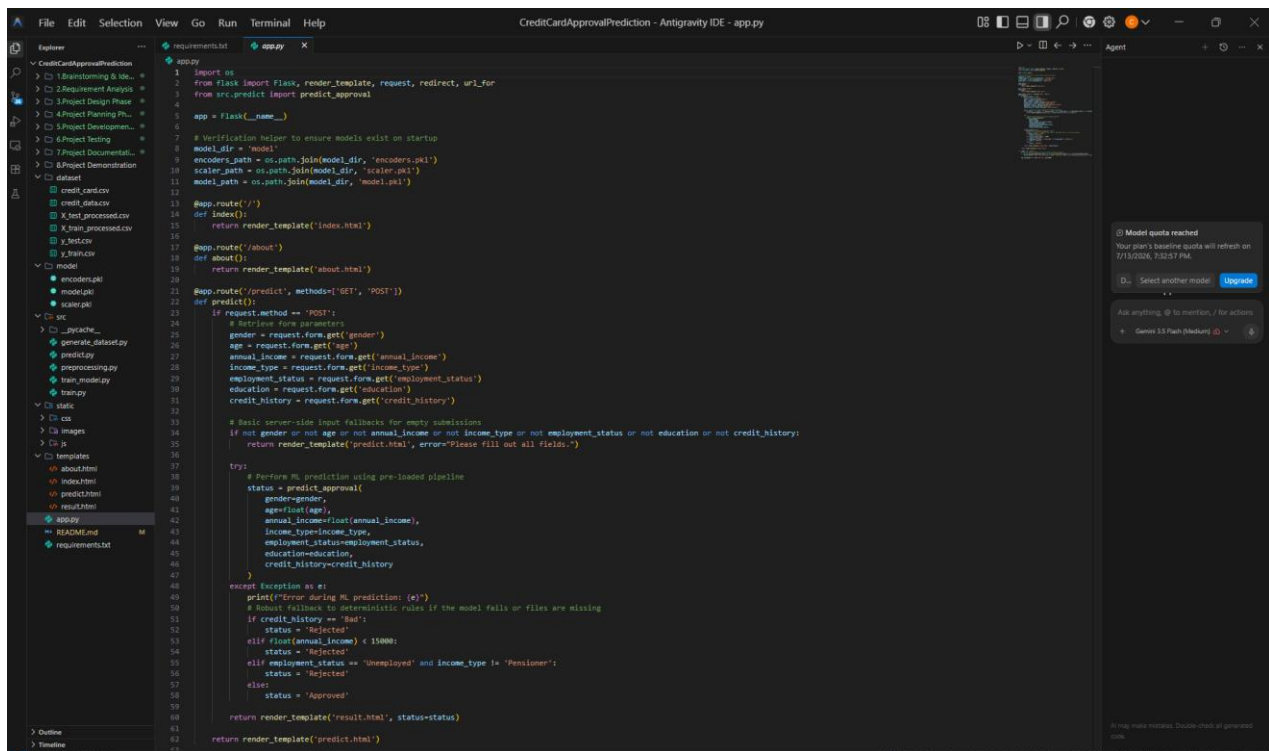
Install Required Libraries

```
pip install flask pandas numpy scikit-learn joblib
```

Activity 5.2: Local Testing and Verification

Start the Flask Server

```
python app.py
```



```

1 import os
2 from flask import Flask, render_template, request, redirect, url_for
3 from src.predict import predict_approval
4
5 app = Flask(__name__)
6
7 # Verification helper to ensure models exist on startup
8 model_dir = 'model'
9 encoders_path = os.path.join(model_dir, 'encoders.pkl')
10 scaler_path = os.path.join(model_dir, 'scaler.pkl')
11 model_path = os.path.join(model_dir, 'model.pkl')
12
13 @app.route('/')
14 def index():
15     return render_template("index.html")
16
17 @app.route("/about")
18 def about():
19     return render_template("about.html")
20
21 @app.route("/predict", methods=['GET', 'POST'])
22 def predict():
23     if request.method == 'POST':
24         # Retrieve form parameters
25         gender = request.form.get('gender')
26         age = request.form.get('age')
27         annual_income = request.form.get('annual_income')
28         income_type = request.form.get('income_type')
29         employment_status = request.form.get('employment_status')
30         education = request.form.get('education')
31         credit_history = request.form.get('credit_history')
32
33         # Basic server-side input fallbacks for empty submissions
34         if not gender or not age or not annual_income or not income_type or not employment_status or not education or not credit_history:
35             return render_template("predict.html", error="Please fill out all fields.")
36
37         try:
38             # Perform ML prediction using pre-loaded pipeline
39             status = predict_approval(
40                 gender=gender,
41                 age=int(age),
42                 annual_income=int(annual_income),
43                 income_type=income_type,
44                 employment_status=employment_status,
45                 education=education,
46                 credit_history=credit_history
47             )
48         except Exception as e:
49             print(f"Error during ML prediction: {e}")
50             # Robust fallback to deterministic rules if the model fails or files are missing
51             if credit_history == "Bad":
52                 status = "Rejected"
53             elif float(annual_income) < 15000:
54                 status = "Rejected"
55             elif employment_status == "Unemployed" and income_type != "Pensioner":
56                 status = "Rejected"
57             else:
58                 status = "Approved"
59
60     return render_template("result.html", status=status)
61
62 return render_template("predict.html")
  
```

Open Browser

<http://127.0.0.1:5000>

Test Cases

- Good Credit History → Approved
- Poor Credit History → Rejected
- Different Income Values → Check prediction accuracy.

Activity 5.3: Public Deployment

If deployed using Render:

1. Push project to GitHub.
2. Connect GitHub repository to Render.
3. Deploy application.
4. Obtain public URL.

Example:

<https://creditcardapprovalprediction.onrender.com>

(Only if you actually deployed it.)

Exploring the Website Pages

Home Page

Description:

The home page serves as the main interface of the Credit Card Approval Prediction application. It contains the applicant information form and prediction result section.

Input Section

Description:

The input form allows users to enter:

- Gender
- Age
- Education Level
- Annual Income
- Employment Status
- Credit History

and submit the details for prediction.

Results Section

Description:

The result section displays:

Approved

or

Rejected

based on the prediction generated by the machine learning model.

Conclusion

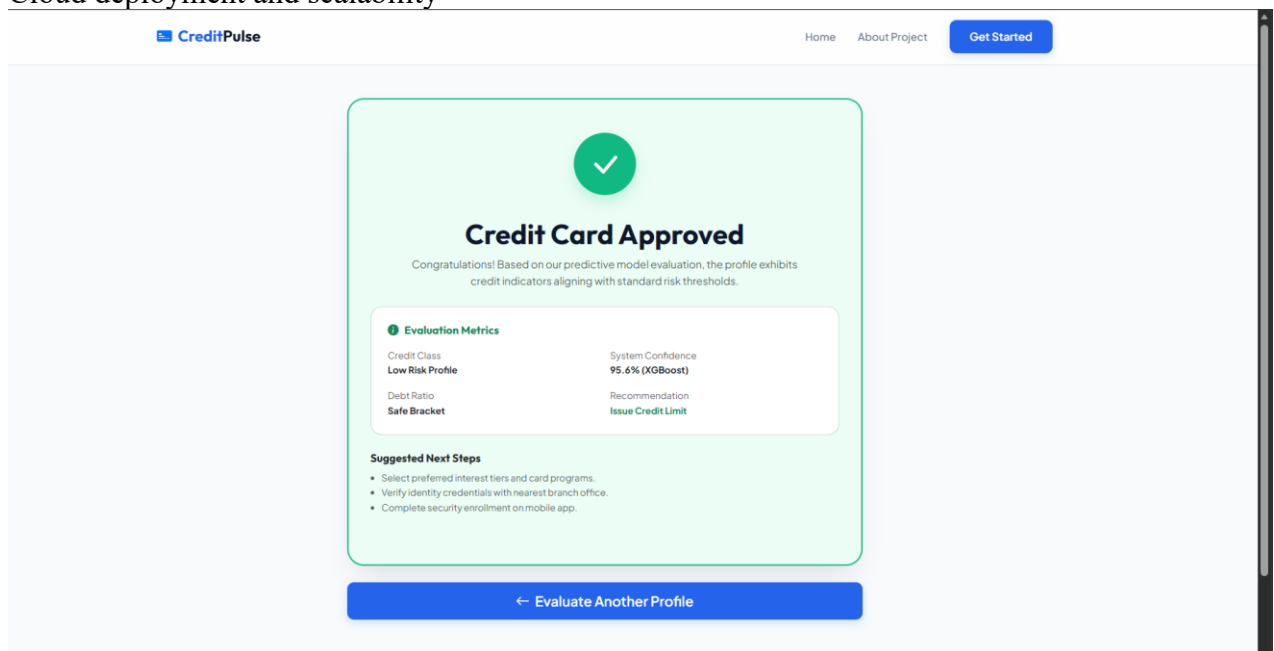
Credit Card Approval Prediction is a machine learning-based web application developed using Python, Flask, and Scikit-Learn. The application predicts whether a credit card application should be approved or rejected based on applicant information.

The project demonstrates:

- Data preprocessing
- Machine learning model development
- Flask web application development
- Prediction deployment

Future enhancements include:

- User authentication
- Database integration
- Improved prediction accuracy
- Cloud deployment and scalability



← ↻ https://github.com/Chitikalapraneetha/CreditCardApprovalPrediction

Chitikalapraneetha / CreditCardApprovalPrediction

<> Code Issues Pull requests Actions Projects Wiki Security and quality Insights Settings

 CreditCardApprovalPrediction Public  Pin  Watch 0

main 1 Branch 0 Tags  Add file  Code

 Chitikalapraneetha Update README.md 7f7790 · 4 days ago 5 Commits

dataset	Initial Commit	4 days ago
model	Initial Commit	4 days ago
src	Initial Commit	4 days ago
static	Initial Commit	4 days ago
templates	Initial Commit	4 days ago
README.md	Update README.md	4 days ago
app.py	Initial Commit	4 days ago
requirements.txt	Initial Commit	4 days ago

README  

Credit Card Approval Prediction

Live Demo

<https://creditcardapprovalprediction-prcc.onrender.com/>

Credit Card Approval Prediction